

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Основы алгоритмизации и программирования

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему

ИГРОВОЕ ПРОГРАММНОЕ СРЕДСТВО ГРАФИЧЕСКИЕ АССОЦИИ

БГУИР КР 6-05-0612-01 107 ПЗ

Студент: гр. 451001 Држевецкий Н. А.

Руководитель:
асс. Фадеева Е.Е.

Минск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРОТОТИПОВ, ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОЕКТИРУЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ	6
1.1 Примеры решения аналогичных задач	6
1.2 Требования к проектируемому программному средству.....	7
2 АНАЛИЗ ТРЕБОВАНИЙ К ПС И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	9
2.1 Описание функциональности ПС.....	9
2.2 Спецификация функциональных требований	11
3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА	14
3.1 Разработка алгоритма ПС	14
3.2 Алгоритмы отдельных модулей	14
4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА....	22
4.1 Структура программного средства	22
4.2 Описание логики, процедур и функций.....	22
5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ	28
5.1 Тестирование программного средства.....	28
5.2 Экспериментальная проверка.....	31
5.3 Анализ полученных данных	31
5.4 Результаты тестирования.....	31
6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ	33
6.1 Установка приложения “Battle Of Copies”	33
6.2 Эксплуатация приложения “Battle Of Copies”	34
ЗАКЛЮЧЕНИЕ	39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	40
ПРИЛОЖЕНИЯ.....	41
ВЕДОМОСТЬ	62

ВВЕДЕНИЕ

Современные игровые технологии охватывают широкий спектр задач, включая не только развлекательные, но и образовательные и коммуникативные функции. Особенно популярными становятся игры, сочетающие элементы творчества и взаимодействия между участниками, способствующие развитию воображения, нестандартного мышления и способности к визуальной интерпретации.

Одним из таких форматов являются многопользовательские игры, основанные на рисовании и угадывании, примером которых служит известная онлайн-игра Copyrighted Artists[1]. Целью данной курсовой работы стало создание аналогичного программного средства, предназначенного для локального использования на одном устройстве, без подключения к сети Интернет. Это позволяет адаптировать игру под оффлайн-среду – учебную аудиторию, мероприятие или домашнее использование.

Программное средство реализовано на языке программирования Delphi с использованием библиотеки VCL, что позволяет быстро и удобно создавать визуальные интерфейсы и управлять графическими компонентами, необходимыми для этапов рисования, голосования и отображения результатов.

В ходе выполнения курсовой работы были поставлены задачи:

- 1 Анализ существующих прототипов и формулировка требований к собственной реализации.
- 2 Проектирование логики игры и пользовательского интерфейса.
- 3 Разработка программного кода, реализующего пошаговые этапы игры.
- 4 Тестирование работоспособности программного средства в различных сценариях.
- 5 Документирование архитектуры, алгоритмов и результатов.

Разработка данной программы представляет собой не только реализацию творческого игрового механизма, но и практическую задачу по проектированию, созданию и проверке комплексного пользовательского программного продукта. Результатом работы является полностью функционирующее приложение, соответствующее функциональным требованиям и пригодное к реальному использованию.

1 АНАЛИЗ ПРОТОТИПОВ, ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОЕКТИРУЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ

1.1 Примеры решения аналогичных задач

«Copyrighted Artists» – многопользовательская онлайн-игра, разработанная на платформе Roblox командой Rawblocky, в которой игроки рисуют изображения и пытаются отличить оригинал от подделки[2]. Правила игры:

1 В начале раунда каждому игроку дается случайная фраза, по теме которой требуется нарисовать изображение.

2 После завершения оригинального этапа каждый игрок случайным образом получает рисунок другого участника и должен его срисовать, не зная оригинальной фразы.

3 Затем все рисунки (оригиналы и копии) появляются на экране, и игроки голосуют за то, какой из них настоящий.

4 Очки начисляются за правильные догадки и за то, что другие игроки ошибочно выбрали подделку.

5 Побеждают игроки с наибольшим количеством баллов.

Достоинства:

1 Бесплатный доступ.

2 Многопользовательская игра.

3 Подходит для всех возрастов.

4 Развитие внимательности.

5 Развитие способности рисования.

Недостатки:

1 Необходимо подключение к интернету.

2 Необходима регистрация аккаунта.

3 Нет поддержки русского языка.

4 Долгое ожидание раундов.

«Drawful» – многопользовательская онлайн-игра, разработанная компанией Jackbox Games, в которой игроки рисуют по странному или забавному заданию картинки и пытаются угадать, что именно изображено[3]. Правила игры:

1 В начале раунда каждому игроку дается случайная фраза, по теме которой требуется нарисовать изображение.

2 После завершения рисунки появляются на экране, и остальные участники придумывают к ним подписи.

3 Далее все подписи (включая правильную) показываются на экране, и игроки голосуют за наиболее правдоподобный вариант.

4 Очки начисляются за правильные догадки и за то, что другие игроки выбирают придуманные ими ложные подписи.

5 Побеждает игрок с наибольшим количеством баллов.

Достоинства:

1 Подходит для всех возрастов.

2 Многопользовательская игра.

3 Мгновенное начало раундов.

4 Регистрация аккаунта не требуется.

5 Развитие сообразительности.

6 Развитие способности рисования.

Недостатки:

1 Необходимо подключение к интернету.

2 Платный доступ.

3 Нет поддержки русского языка.

1.2 Требования к проектируемому программному средству

1.2.1 Назначение разработки.

Разрабатываемое программное средство представляет собой локальную версию игры, основанной на механике "Copyrighted Artists". Цель разработки – создание игровой программы, позволяющей нескольким игрокам на одном устройстве участвовать в раундах рисования, копирования и угадывания оригинального изображения. Программа предназначена для развлечения и тренировки творческих навыков пользователей.

1.2.2 Состав выполняемых функций.

1 Генерация случайных заданий для рисования.

2 Обеспечение графического интерфейса для рисования с базовыми инструментами.

3 Разрешение рисования на выбранный промежуток времени.

4 Временное сохранение рисунков пользователей для последующего сравнения.

5 Передача управления игроком другому игроку в соответствующих раундах.

6 Демонстрация оригиналов и копий с возможностью голосования.

7 Подсчет очков, определение победителей.

8 Отображение итоговых результатов игры.

1.2.3 Входные данные.

- 1 Число игроков.
- 2 Прозвища игроков.
- 3 Время на рисование.
- 4 Рисунки, созданные игроками в ходе игры.
- 5 Выбор игроков при голосовании за оригинальный рисунок.

1.2.4 Выходные данные.

- 1 Отображение заданий для рисования.
- 2 Отображение рисунков игроков.
- 3 Итоговые результаты голосования.
- 4 Общий рейтинг игроков по количеству набранных очков.
- 5 Графический интерфейс для удобного взаимодействия с программой.

1.2.5 Требования к временным характеристикам.

- 1 Генерация нового задания – не более 1 секунды.
- 2 Итоговый подсчет очков и составление рейтинга игроков – не более 1 секунды.
- 3 Отображение сохраненных рисунков – не более 1 секунды.

1.2.6 Требования к надежности.

- 1 Корректная обработка некорректного ввода
- 2 Предотвращение сбоев при использовании графических инструментов.

1.2.7 Условия эксплуатации.

Программа предназначена для работы на персональном компьютере (без необходимости подключения к интернету).

Поддерживаемые операционные системы:

- 1 Windows 11 (32-bit, 64-bit).
- 2 Windows 10 (32-bit, 64-bit).
- 3 Windows 7 (32-bit, 64-bit).

Управление осуществляется с помощью клавиатуры и мыши.

2 АНАЛИЗ ТРЕБОВАНИЙ К ПС И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

2.1 Описание функциональности ПС

Всевозможные акторы:

2.1.1 Выбор количества игроков, ввод псевдонимов и времени на рисование.

Акторы: игрок, программа.

Цель: определение количества участников и фиксирование их имен в системе.

Предусловия: программа запущена, игра находится в стартовом меню.

Основной сценарий:

1 Игрок вводит количество участников (от 3).

2 Игроки вводят свои псевдонимы.

3 Игрок вводит количество секунд на рисование.

4 Программа сохраняет данные и переходит к следующему этапу.

Альтернативные сценарии:

1 Игрок оставляет поле псевдонима пустым или вводит псевдоним использующийся другим игроком – программа просит ввести имя повторно.

2 Игрок ставит время на рисование за пределы допустимых значений или оставляет поле пустым – ставится стандартное время.

2.1.2 Генерация задания.

Акторы: программа.

Цель: определение тем для рисования.

Предусловия: все игроки зарегистрированы.

Основной сценарий:

1 Программа случайным образом выбирает уникальную тему для каждого игрока поочередно.

2 Тема отображается на экране на протяжении рисования оригинального рисунка.

Альтернативные сценарии:

1 Файл с заданиями поврежден или отсутствует – предупреждение пользователя.

2.1.3 Этап рисования оригинальных рисунков.

Акторы: игрок, программа.

Цель: создание оригинального рисунка по заданной теме.

Предусловия: игрок получил тему для рисования.

Основной сценарий:

1 Программа отображает поле для рисования и полученную уникальную тему рисунка.

2 Игрок рисует изображение за отведенное время.

3 Программа отслеживает оставшееся время и отображает таймер.

4 По завершении времени программа автоматически сохраняет рисунок и переходит к следующему этапу.

Альтернативные сценарии:

1 При досрочном завершении этапа рисования игроком программа автоматически сохраняет рисунок и переходит к следующему этапу.

2.1.4 Этап рисования копий рисунков.

Актors: игрок, программа.

Цель: создание копии чужого рисунка без знания его темы.

Предусловия: завершен этап рисования оригиналов.

Основной сценарий:

1 Программа случайным образом раздает и отображает каждому игроку рисунок другого игрока на определенное время.

2 По завершении времени программа отображает поле для рисования.

3 Игрок пытается нарисовать максимально похожую на оригинал копию за отведенное время.

4 Программа отслеживает оставшееся время и отображает таймер.

5 По завершении времени программа автоматически сохраняет рисунок и переходит к следующему этапу.

Альтернативные сценарии:

1 При досрочном завершении этапа рисования игроком программа автоматически сохраняет рисунок и переходит к следующему этапу.

2.1.5 Этап голосования.

Актors: игрок, программа.

Цель: выбор оригинального рисунка среди представленных.

Предусловия: все рисунки (оригиналы и копии) загружены в систему.

Основной сценарий:

1 Программа отображает все рисунки (оригиналы и копии) в случайном порядке, не указывая который из них оригинал.

2 Каждый игрок, чьи рисунки на данный момент не отображены, выбирает тот рисунок, который, по его мнению, является оригиналом.

3 После завершения голосования программа переходит к следующему этапу.

Альтернативные сценарии:

1 Игрок не выбирает ни один из вариантов – программа фиксирует пропущенный голос и не выдает ему очки.

2.1.6 Подсчет очков и отображение результатов.

Акторы: программа.

Цель: определение набранных очков за раунд.

Предусловия: голосование завершено, все голоса подсчитаны.

Основной сценарий:

1 Программа анализирует результаты голосования.

2 Игроки получают очки за правильное угадывание оригинального рисунка.

3 Игроки получают дополнительные очки, если их подделка была выбрана другими участниками.

4 Итоговая таблица с очками игроков отображается на экране.

5 Программа переходит к следующему раунду.

Альтернативные сценарии:

1 Несколько игроков набрали одинаковое количество очков – программа фиксирует ничью в таблице.

2.1.7 Определение победителя и завершение игры.

Акторы: программа.

Цель: определить победителя после завершения всех раундов.

Предусловия: все запланированные раунды завершены.

Основной сценарий:

1 Программа анализирует общий рейтинг игроков.

2 Победителем объявляется игрок с наибольшим количеством очков.

3 Программа предлагает игрокам начать новую игру или выйти из программы.

Альтернативные сценарии:

1 Несколько игроков набрали одинаковое количество очков – программа объявляет нескольких победителей.

2.2 Спецификация функциональных требований

2.2.1 Выбор количества игроков, ввод псевдонимов и времени на рисование.

1 Пользователь должен иметь возможность выбрать количество игроков (от 3).

2 Интерфейс должен предусматривать поля для ввода псевдонимов всех игроков.

3 Введенные псевдонимы должны быть уникальными, программа должна проверять на дубли.

4 Пользователь должен ввести время на рисование в пределах допустимого диапазона.

5 Если пользователь оставил поле времени пустым или ввел некорректное значение, программа должна установить стандартное время.

6 После ввода всех данных программа должна сохранять их в память до завершения игры.

2.2.2 Генерация задания.

1 Программа должна выбирать случайное задание для каждого игрока поочередно из предустановленного списка.

2 Программа должна гарантировать, что одно задание не может выпасть двум игрокам в одном раунде.

3 Тема должна всегда отображаться во время этапа рисования оригинального рисунка.

4 Если файл с заданиями отсутствует или поврежден, программа должна уведомить игрока об этом.

2.2.3 Этап рисования оригинальных рисунков.

1 Игрок должен иметь возможность и доступ к полю для рисования.

2 Тема должна быть всегда видна во время рисования.

3 Программа должна отображать таймер с обратным отсчетом времени.

4 По истечении времени программа автоматически завершает этап рисования и сохраняет результат.

5 Игрок может досрочно завершить рисование, нажав соответствующую кнопку, после чего программа сразу сохранит результат и перейдет к следующему этапу.

2.2.4 Этап рисования копий рисунков.

1 Программа случайным образом раздает каждому игроку оригинальный рисунок другого игрока.

2 Игрок не должен видеть тему оригинального рисунка.

3 Игрок должен иметь возможность и доступ к полю для рисования, как на этапе рисования оригинала.

4 По завершении времени программа автоматически завершает этап и сохраняет рисунок.

5 Игрок может досрочно завершить рисование, нажав соответствующую кнопку, после чего программа сразу сохранит результат и перейдет к следующему этапу.

2.2.5 Этап голосования.

1 Программа должна отображать оригинальный рисунок и его копии в случайном порядке, не указывая, какой из них оригинал.

2 Игроки, чьи рисунки не отображены в этом раунде, выбирают тот, который, по их мнению, является оригиналом.

3 Если игрок не выбрал ни один вариант – программа фиксирует пропущенный голос и не начисляет ему очки.

4 После завершения голосования программа должна перейти к следующему этапу.

2.2.6 Подсчет очков и отображение результатов.

1 Программа анализирует результаты голосования.

2 Игроки получают определенное количество очков за правильное угадывание оригинала.

3 Игроки получают дополнительные очки, если их подделка была выбрана другими участниками.

4 Программа должна корректно суммировать очки за все предыдущие раунды.

5 Таблица рейтинга должна отображать всех игроков и их текущие очки.

2.2.7 Определение победителя и завершение игры.

1 После завершения всех раундов программа должна определить игрока с наибольшим количеством очков.

2 В случае ничьей программа должна объявить нескольких победителей.

3 После объявления победителя программа должна предложить выйти в главное меню.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Разработка алгоритма ПС

Алгоритм работы программы можно представить в виде последовательности действий:

- 1 Регистрация игроков.
- 2 Генерация заданий.
- 3 Этап рисования оригиналов.
- 4 Этап рисования копий.
- 5 Этап голосования.
- 6 Подсчет очков.
- 7 Определение победителя.

3.2 Алгоритмы отдельных модулей

3.2.1 Алгоритм регистрации игроков и генерации заданий

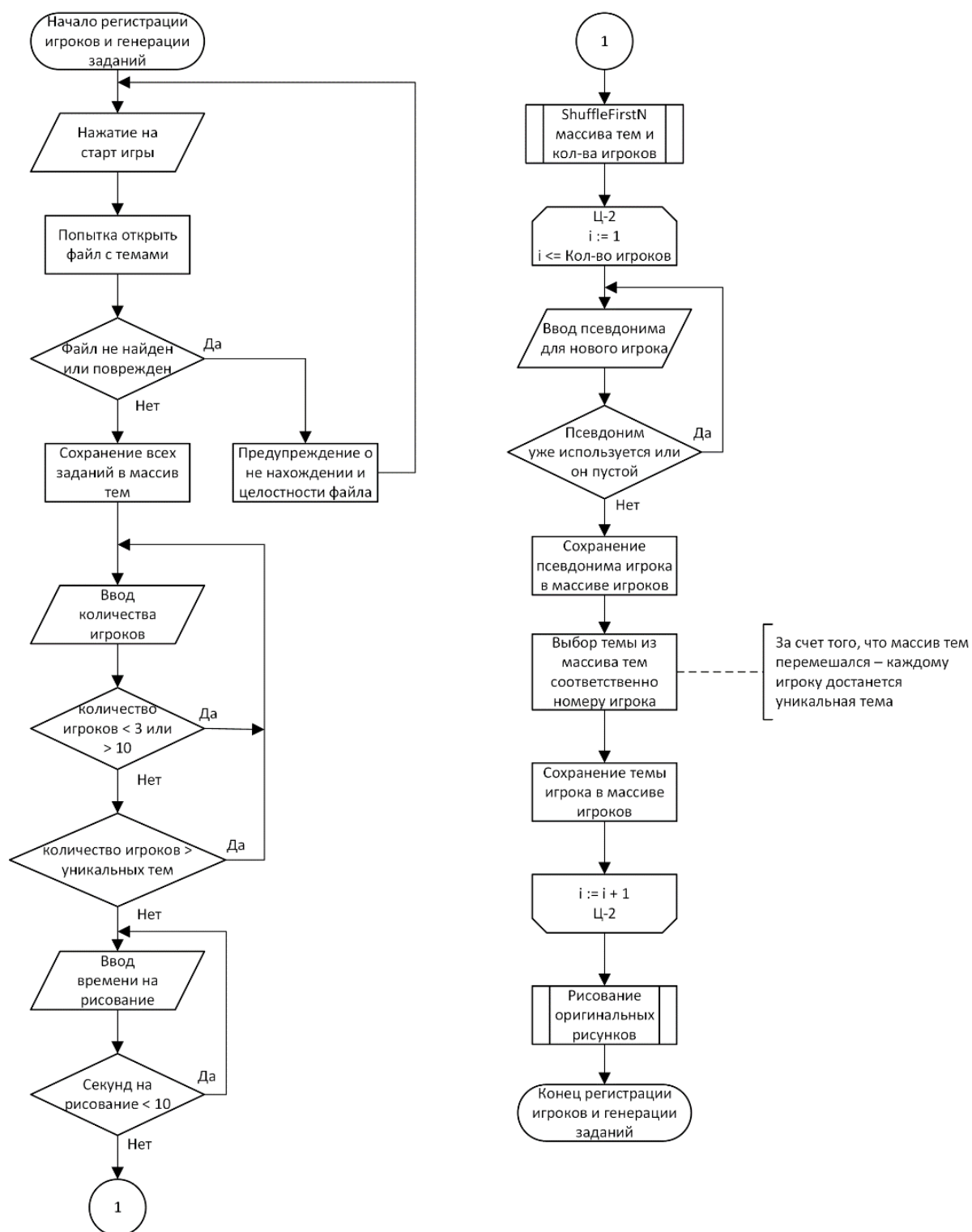


Рисунок 3.1 – Регистрация игроков и генерация заданий



Рисунок 3.2 – Функция ShuffleFirstN

3.2.2 Алгоритм рисования оригинальных рисунков

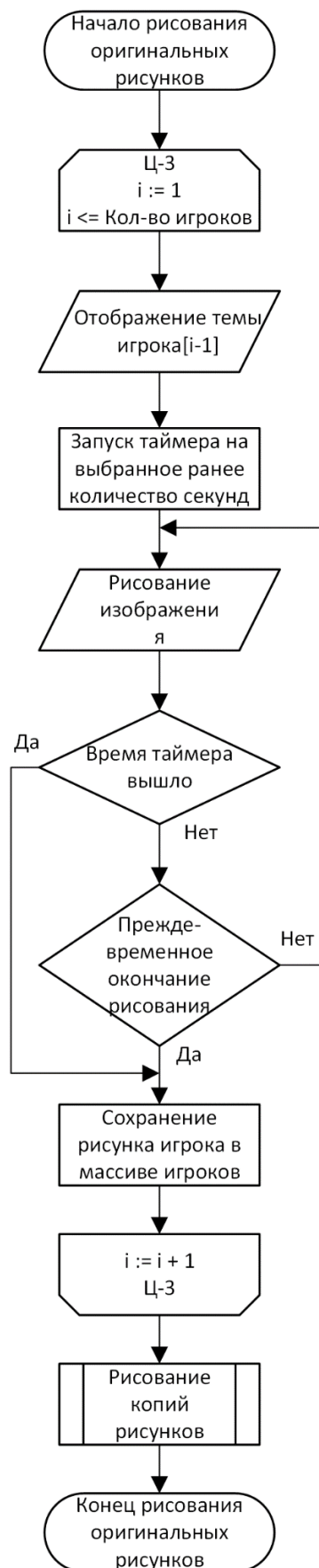


Рисунок 3.3 – Рисование оригинальных рисунков

3.2.3 Алгоритм рисования копий рисунков

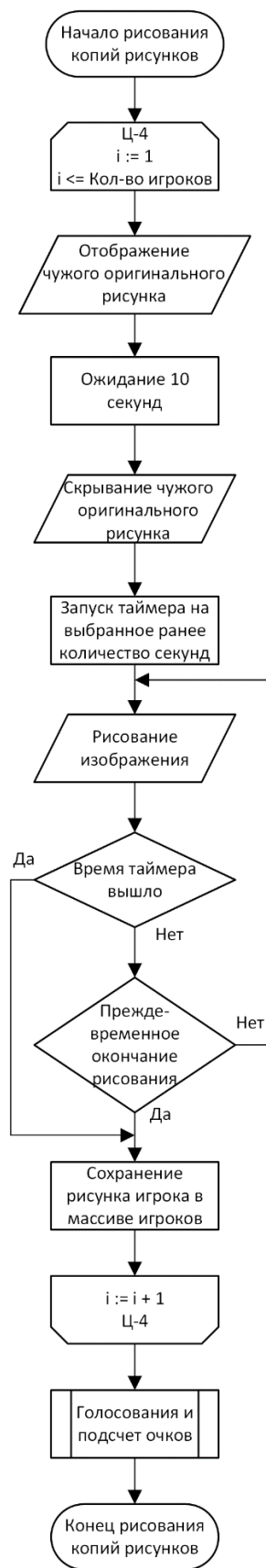


Рисунок 3.4 – Рисование копий рисунков

3.2.4 Алгоритм голосования и подсчета очков

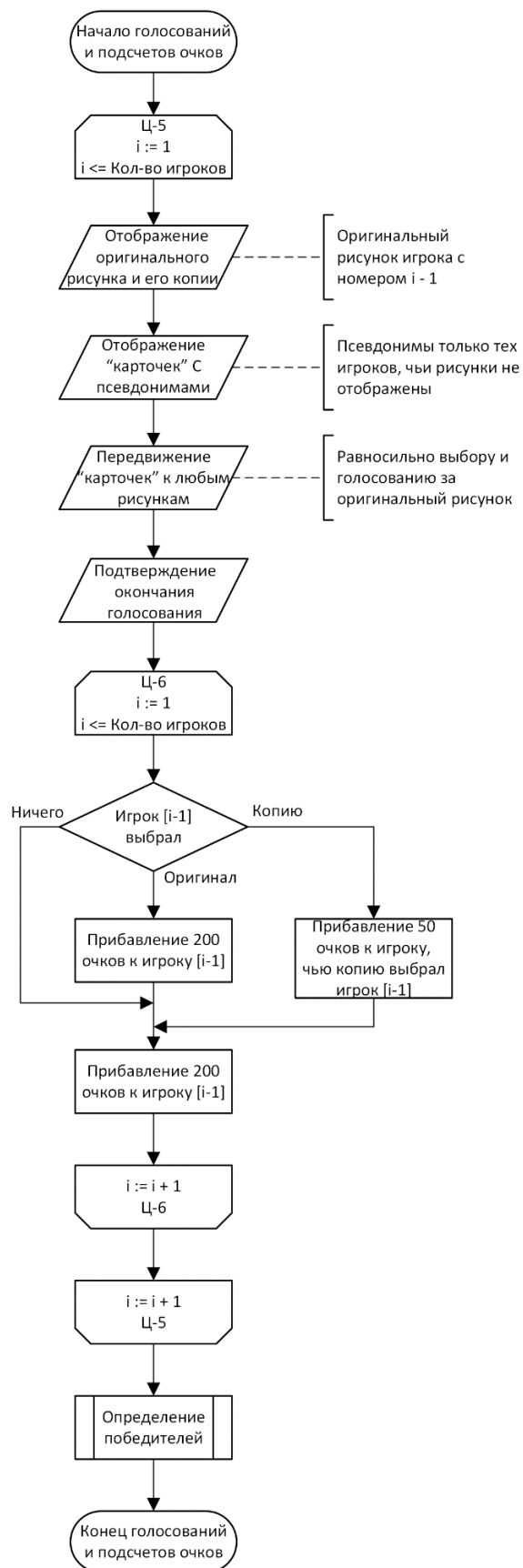


Рисунок 3.5 – Голосование и подсчет очков

3.2.5 Алгоритм определения победителей



Рисунок 3.6 – Определение победителей

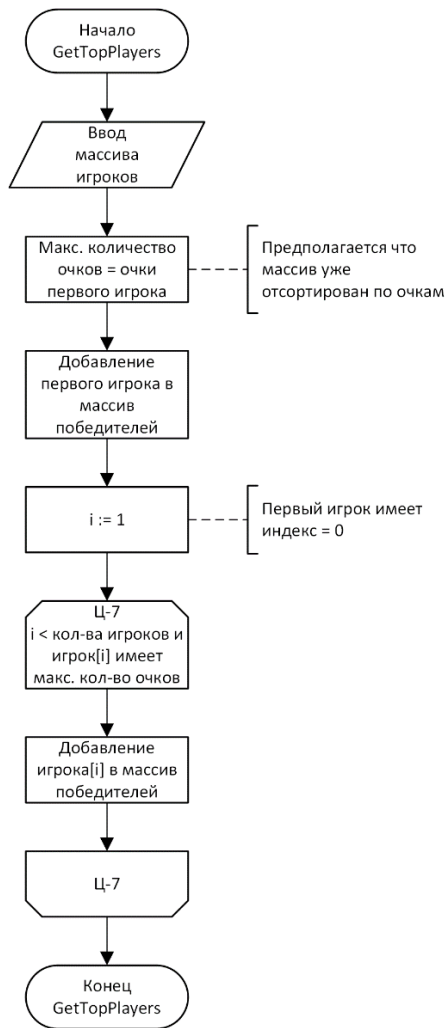


Рисунок 3.7 – Функция GetTopPlayers

4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА

4.1 Структура программного средства

На основании ранее спроектированных алгоритмах осуществляется непосредственная реализация программного средства. Выполняется программирование всех основных модулей, их отладка и сборка. Разрабатывается структура программы, включающая описание процедур, функций и взаимодействия компонентов.

Архитектура построена по модульному принципу и состоит из следующих модулей:

1 Модуль GameLogic – содержит основную игровую логику рисования, переход между игроками и стадиями игры.

2 Модуль VoteLogic – обрабатывает стадию голосования, подсчитывает голоса и соответствующие им игровые очки.

3 Модуль SetRulesLogic – обеспечивает настройку параметров перед игрой, регистрацию игроков.

4 Модуль PanelsLogic – обеспечивает переход из одной панели к другой через скрывание всех панелей на форме и отображении нужной.

5 Модуль WinLogic – отвечает за обработку конечных результатов и отображение финальной таблицы победителей.

4.2 Описание логики, процедур и функций

4.2.1 Основные типы и структуры данных.

В модуле SetRulesLogic определён основной тип данных: TPlayer, обозначающий одного из игроков. Он содержит:

1 Name: String – псевдоним игрока.

2 Prompt: String – задание игрока.

3 id, copyid: Integer – номера данного игрока и игрока, копирующего рисунок соответственно.

4 Score: Integer – количество очков.

5 BitmapOrig, BitmapCopy: TBitmap – рисунки данного игрока и игрока, копирующего рисунок соответственно.

Игроки хранятся в динамическом массиве PlayersList: array of TPlayer

В модуле VoteLogic определены структуры для голосования: TCard – обозначающий карточку для голосования одного из игроков, TBallot – обозначающий бюллетень на голосование. TCard содержит:

1 Player: PPlayer (где PPlayer = ^TPlayer) – игрок, кому принадлежит карточка.

2 LabelCard: TLabel – сама карточка в виде TLabel.

3 SelectedId: Integer – номер игрока, за которого проголосовали.

TBallot содержит:

1 Player: PPlayer (где PPlayer = ^TPlayer) – игрок, кому принадлежит бюллетень.

2 Bitmap: TBitmap – рисунок данного игрока.

3 PaintVote: TPaintBox – место отображения данного рисунка.

4 OrigOrCopyText: TStaticText – итоговый текст обозначающий оригинал или копию.

4.2.2 Подпрограммы модуля GameLogic.

Таблица 4.1 – Описание процедур и функций модуля GameLogic

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
StartGame	Инициализирует игру, создаёт холст, запускает первого игрока	procedure StartGame;	–	–
LoadPaint	Загружает текущее изображение на канву	procedure LoadPaint;	–	–
CreateBitmap	Создаёт основной холст для рисования	procedure CreateBitmap;	–	–
CleanBitmap	Очищает переданный холст белым цветом	procedure CleanBitmap(var CleanThisBM: TBitmap);	CleanThisBM	Изображение, которое требуется очистить
PaintMouseDown	Инициализирует начало линии рисования при нажатии мыши	procedure PaintMouseDown(X, Y: Integer);	X, Y	Координаты начала линии
PaintMouseMove	Рисует линию от предыдущей точки до текущей при движении мыши	procedure PaintMouseMove(X, Y: Integer);	X, Y	Координаты текущего положения мыши

Продолжение таблицы 4.1

PaintMouseUp	Завершает рисование и сохраняет состояние для отмены	procedure PaintMouseUp;	—	—
SaveUndoBitmap	Сохраняет текущее изображение в стек отмен	procedure SaveUndoBitmap;	—	—
SetUndoBitmap	Откатывает последнее изменение рисунка	procedure SetUndoBitmap;	—	—
HidePaintBox	Управляет видимостью и доступностью канвы и подсказок	procedure HidePaintBox(hide: Boolean);	hide	True — скрыть, False — показать
GameTimer	Обновляет прогресс-бар и завершает таймер при достижении лимита	procedure GameTimer;	—	—
StartTimerPB	Запускает таймер с визуальным прогрессом и колбэком после окончания	procedure StartTimerPB(Duration: Integer; AfterProc: TProc);	Duration, AfterProc	Время таймера (мс); процедура, вызываемая по завершении
NextPlayerWaitPrompt	Показывает экран ожидания с инструкцией перед этапом игрока	procedure NextPlayerWaitPrompt(const PromptCaption, TempCaption, Hint: string; AfterProc: TProc);	PromptCaption, TempCaption, Hint, AfterProc	Текст подсказки, временный текст, текст помощи, завершение
NextPlayerWaitReady	Загружает форму ожидания готовности следующего игрока	procedure NextPlayerWaitReady;	—	—
NextPlayerRemember	Отображает оригинальный рисунок для запоминания	procedure NextPlayerRemember;	—	—
NextPlayerGame	Подготавливает интерфейс и состояние для следующего игрока	procedure NextPlayerGame;	—	—

Продолжение таблицы 4.1

AfterGame	Сохраняет результат этапа, переключает игроков и запускает голосование	procedure AfterGame;	—	—
PlayerIsReady	Обрабатывает нажатие «Готов» и переводит к рисованию или запоминанию	procedure PlayerIsReady;	—	—

4.2.3 Подпрограммы модуля VoteLogic.

Таблица 4.2 – Описание процедур и функций модуля VoteLogic

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
VoteStart	Инициализирует стадию голосования	procedure VoteStart;	—	—
SelectCard	Обрабатывает выбор карточки игрока	procedure SelectCard(Sender: TObject);	Sender	Компонент, вызвавший событие (обычно карточка)
PaintVoteSelect	Обрабатывает выбор варианта голосования	procedure PaintVoteSelect(Sender: TObject);	Sender	Компонент, по которому произведено нажатие
CalculatePoints	Подсчитывает и распределяет очки между игроками	procedure CalculatePoints;	—	—
LoadPaintVote	Перерисовывает рисунок	procedure LoadPaintVote(PaintVote: TPaintBox; Bitmap: TBitmap);	PaintVote, Bitmap	Место и ссылка на изображение, которое требуется перерисовать
CreateNewCard	Создаёт визуальные карточки для голосования	procedure CreateNewCard(var Player: TPlayer; VotersCount: Integer);	Player, VotersCount	Игрок и его номер карточки

Продолжение таблицы 4.2

ShowOrig AndCopy	Показывает, где оригинал, а где копия	procedure ShowOrigAndCopy;	—	—
NextPlayer Vote	Переход к следующим игрокам на голосование	procedure NextPlayerVote;	—	—

4.2.4 Подпрограммы модуля SetRulesLogic.

Таблица 4.3 – Описание процедур и функций модуля SetRulesLogic

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
AddNewPlayer	Добавляет нового игрока в список игроков	procedure AddNewPlayer;	—	—
AddNewRules	Сохраняет введённые пользователем параметры игры	procedure AddNewRules;	—	—
LoadPrompts	Загружает список заданий (prompt'ов) из файла или по умолчанию	procedure LoadPrompts;	—	—
ShuffleFirstN	Расставляет первые N значений массива в случайном порядке	procedure ShuffleFirstN(var SomeList: TPromptsList; N: Integer);	SomeList, N	Массив и сколько первых значений надо перемешать

4.2.5 Подпрограммы модуля PanelsLogic.

Таблица 4.4 – Описание процедур и функций модуля PanelsLogic

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
---------------------	----------	-----------	------------------	-------------------------

Продолжение таблицы 4.4

ShowPanel	Скрывает все панели и отображает выбранную	procedure ShowPanel(panel: TPanel);	panel	Панель, которую требуется отобразить
-----------	--	-------------------------------------	-------	--------------------------------------

4.2.6 Подпрограммы модуля WinLogic.

Таблица 4.5 – Описание процедур модуля WinLogic

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
SortTopPlayers	Сортирует список игроков по убыванию очков	procedure SortTopPlayers;	—	—
ShowWinTable	Отображает таблицу результатов и победителей	procedure ShowWinTable;	—	—
FreePlayersList	Очищает массив игроков, включая все рисунки	procedure FreePlayersList;	—	—

5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

5.1 Тестирование программного средства

Тестирование проводилось вручную в среде исполнения приложения. Были сформированы сценарии тестов, имитирующие действия пользователей в различных условиях, включая стандартные и обычные случаи. Примеры тестов приведены в таблице 5.1.

Таблица 5.1 – Примеры проведённых тестов

№	Описание теста	Входные данные	Ожидаемый результат	Фактический результат
1	Ввод количества игроков меньше 3	2 игрока	Появляется предупреждение об ошибке, невозможно начать игру	Успешно
2	Ввод пустого имени игрока	Имя = ""	Появляется сообщение «Некорректный псевдоним»	Успешно
3	Ввод одинаковых имён игроков	Имя1 = "Вася", Имя2 = "Вася"	Появляется сообщение «Некорректный псевдоним»	Успешно
4	Установка корректного времени на рисование	Время = 60	Программа принимает значение и сохраняет	Успешно
5	Установка некорректного времени (меньше 10 сек)	Время = 5	Появляется сообщение «Некорректное время»	Успешно
6	Установка некорректного времени (больше 120 сек)	Время = 133	Появляется сообщение «Некорректное время»	Успешно
7	Назначение уникальных заданий	4 игрока	Всем назначаются разные темы	Успешно

Продолжение таблицы 5.1

8	Повреждённый файл заданий	Файл отсутствует или пустой	Программа сообщает об ошибке, игра не начинается	Успешно
9	Этап рисования: игрок завершает досрочно	Нажатие «Закончить» до таймера	Программа сохраняет рисунок и переходит к следующему этапу	Успешно
10	Этап рисования: игрок не рисует вообще	Таймер доходит до нуля, никаких действий	Сохраняется пустой рисунок	Успешно
11	Этап копирования: назначение чужого рисунка	3 игрока	Каждый игрок получает изображение другого игрока, не своё	Успешно
12	Этап копирования: оригинал и копия визуально отличаются	Тестовый случай с простым оригиналом	Проверяется способность игрока максимально точно копировать	Успешно
13	Голосование: игрок перетаскивает карточку на чужой рисунок	Карточка «Вася» → Рисунок А, конец голосования	Регистрируется голос за соответствующий рисунок	Успешно
14	Голосование: игроки рисовавшие картины не получают карточек	Появление карточек на голосование	Программа создает карточки только тех, кто не рисовал ни один рисунок	Успешно
15	Голосование: игрок не проголосовал	Время истекло	Голос фиксируется как пропущенный, очки не начисляются	Успешно
16	Голосование: два игрока голосуют за одну и ту же копию	Два голоса на один рисунок-копию	Обе попытки учитываются, автор копии получает 2*100 бонусных очков	Успешно
17	Подсчет очков: правильный выбор оригинала	Игрок голосует правильно	Игрок получает 200 бонусных очков	Успешно
18	Подсчет очков: чужая копия введена в заблуждение	Игрок выбрал копию	Автор копии получает 100 бонусных очков	Успешно

Продолжение таблицы 5.1

19	Победитель: у одного игрока больше очков	Игрок А — 300 очков, остальные — меньше	Программа объявляет победителя игрока А	Успешно
20	Победитель: ничья по очкам	Игрок А — 400, Игрок Б — 400	Программа объявляет обоих победителями	Успешно
21	Кнопка «Новая игра»	Нажатие после окончания раунда	Программа очищает состояние и возвращается к регистрации	Успешно
22	Проверка очистки PlayersList перед новой игрой	Начало новой игры	Предыдущие игроки и рисунки очищены	Успешно
23	Очистка лейблов из панели результатов	Переход к следующему раунду	Все TLabel удалены перед отображением новых	Успешно
24	Перетаскивание карточек: бросок на нецелевой объект	Бросок на пустую форму	Игнорируется	Успешно
25	Проверка отзывчивости UI при переходе между этапами	Быстрое нажатие кнопок	Интерфейс не зависает, переходы работают корректно	Успешно
26	Проверка стабильности таймера	Ожидание таймера	По истечении времени автоматически выполняется переход	Успешно
27	Проверка отображения темы во время рисования оригинала	Проверка UI	Тема всегда отображается над полем рисования	Успешно
28	Проверка скрытия темы при рисовании копии	Проверка UI	Тема оригинала не видна игроку	Успешно
29	Очистка рисунков между раундами	Новый раунд	Предыдущие изображения не сохраняются, новые создаются	Успешно

30	Максимальное количество игроков	8 игроков	Все функции работают стабильно, UI остаётся удобным	Успешно
31	Попытка начать игру без выбора игроков	Нажатие «Начать» без заполнения данных	Количество игроков выставлено по умолчанию, появляется сообщение «Некорректный псевдоним»	Успешно
32	Отмена последнего действия при рисовании	Рисование + нажата комбинация Ctrl + Z	Последний штрих удаляется, полотно обновляется	Успешно

5.2 Экспериментальная проверка

Для проверки адекватности игрового процесса программа была протестирована в реальных условиях: с участием группы из 3 человек. Результаты показали:

- 1 Игра полностью воспроизводима.
- 2 Все этапы понятны и не вызывают затруднений.
- 3 Механика голосования интуитивна.
- 4 Победитель определяется честно и логично.

5.3 Анализ полученных данных

Все основные функции программного средства показали стабильную и корректную работу. Были зафиксированы следующие важные результаты:

- 1 В процессе регистрации и настройки исключены ошибки, связанные с пустыми или повторяющимися именами.
- 2 Темы для рисунков назначаются уникально и без повторений в пределах раунда.
- 3 Рисование и копирование работают с учётом времени, включая автоматическое завершение по таймеру.
- 4 Подсчёт очков соответствует правилам: начисление за угадывание оригинала и бонус за обман.
- 5 Победитель определяется корректно даже при равенстве очков.
- 6 Интерфейс автоматически очищается между раундами и при начале новой игры

5.4 Результаты тестирования

Тестирование подтвердило соответствие программного средства требованиям, изложенным в техническом задании. Все ключевые функции реализованы и функционируют правильно. Программа устойчива к ошибкам ввода, не допускает сбоев при отсутствии действия игрока, а интерфейс остается отзывчивым и понятным на всех этапах игры.

Были выявлены некоторые незначительные UX-моменты (например, необходимость подсветки при перетаскивании карточек), которые были устранены в процессе отладки.

Таким образом, можно считать, что программное средство прошло успешную проверку и готово к использованию в учебной или демонстрационной среде.

6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ

6.1 Установка приложения “Battle Of Copies”

Для загрузки программного обеспечения необходимо перейти по предоставленной ссылке[4]. Она направляет на официальную страницу проекта на платформе GitHub.

На данной странице требуется найти и скачать архив программы в формате ZIP:

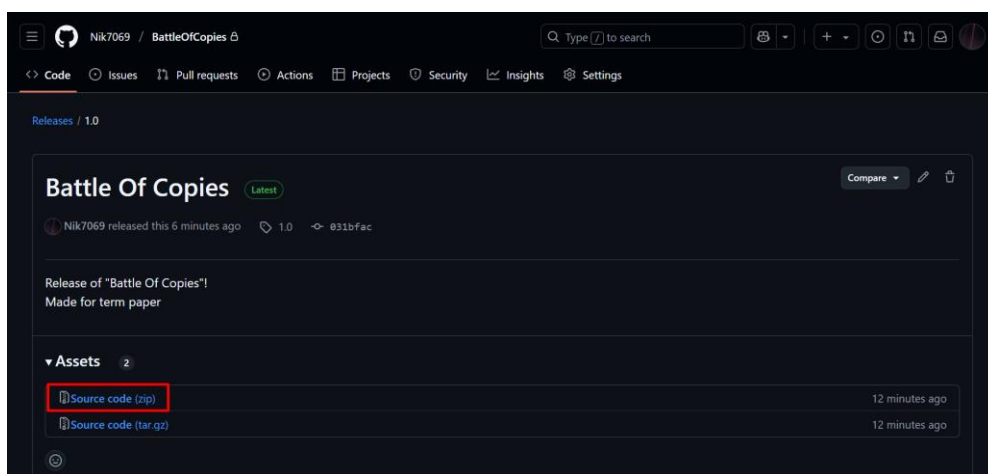


Рисунок 6.1 – Страница для скачивания приложения

После завершения загрузки ZIP-файла его необходимо распаковать в выбранную директорию. В результате разархивации должна быть создана папка, содержащая следующие файлы:

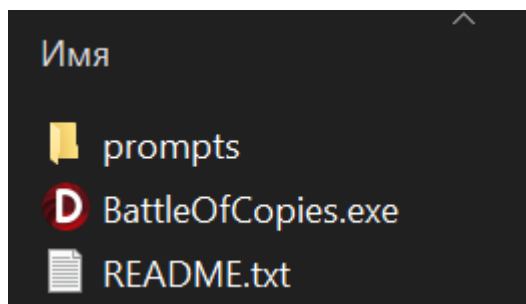


Рисунок 6.2 – Основные файлы программы

Для начала работы с приложением следует открыть распакованную папку и запустить исполняемый файл "BattleOfCopies.exe". После успешного запуска программа будет готова к эксплуатации.

6.2 Эксплуатация приложения “Battle Of Copies”

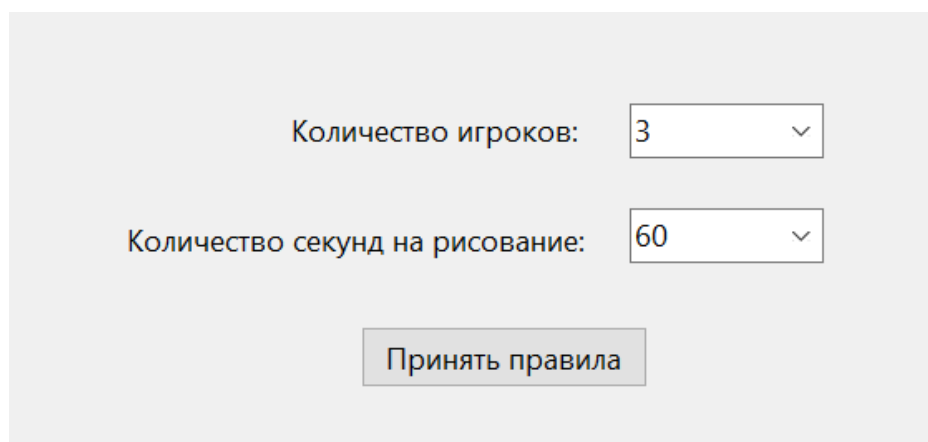
Игра рассчитана на использование на одном устройстве (например, ноутбуке или интерактивной панели) и подходит для коллективной игры без подключения к Интернету.

В меню располагаются кнопки начала игры и выхода из программы. При запуске игры все игроки предположительно обязаны следовать правилам и цели

Каждый игрок должен нарисовать оригинальный рисунок по предложенной теме, а затем попытаться скопировать чужой рисунок, не зная его темы. После этого игроки голосуют, стараясь определить, какой из представленных рисунков является оригиналом. Цель – набрать как можно больше очков, угадывая оригиналы и вводя других игроков в заблуждение своими копиями.

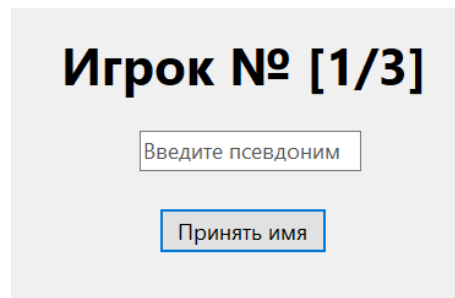
Игру можно условно разделить на несколько этапов. В этапе регистрации игроков есть свои правила:

- 1 Игра требует минимум 3 участников.
- 2 Пользователь вводит общее количество игроков и задаёт имена каждого.
- 3 Также на этом этапе задаётся продолжительность этапа рисования.
- 4 Программа проверяет корректность введённых данных: имена не должны быть пустыми или повторяющимися.
- 5 Программа случайным образом выбирает уникальную тему из заранее заданного списка для каждого игрока.
- 6 Тема назначается так, чтобы не повторяться в пределах одного раунда.
- 7 Игроки поочерёдно получают свои темы при переходе к этапу рисования оригинала.



The image shows a settings window for the game. It has a light gray background. At the top, there is a label "Количество игроков:" followed by a dropdown menu showing the number "3". Below this, there is another label "Количество секунд на рисование:" followed by a dropdown menu showing the number "60". At the bottom center, there is a button with the text "Принять правила".

Рисунок 6.3 – Окно настройки правил игры



Игрок № [1/3]

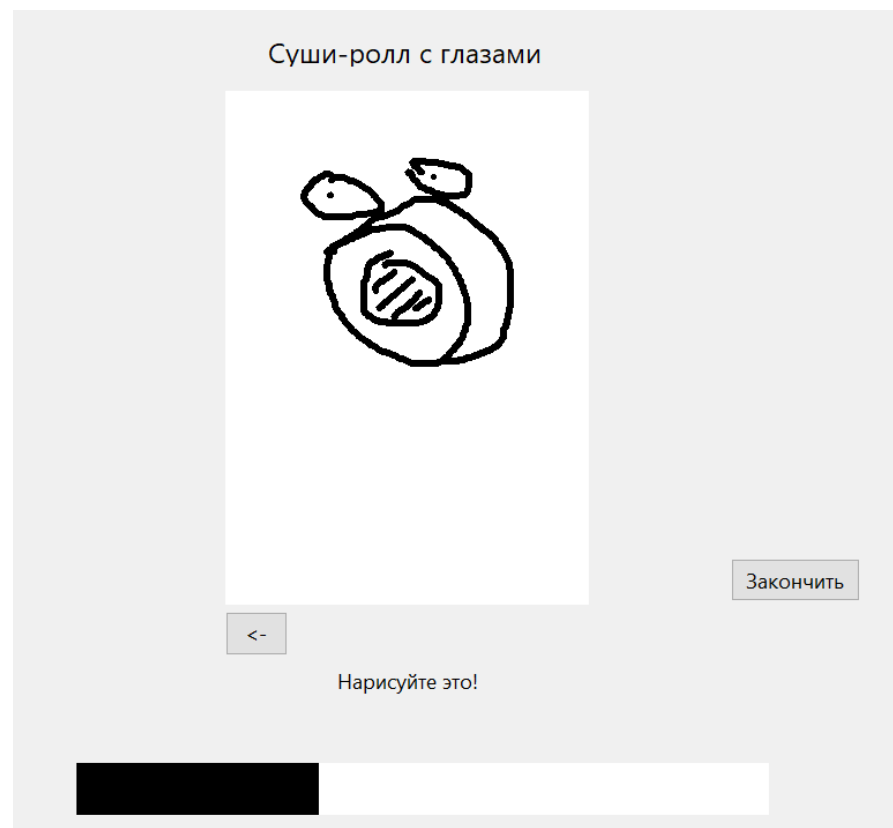
Введите псевдоним

Принять имя

Рисунок 6.4 – Окно регистрации игроков

В этапе рисования оригинальных рисунков также есть свои правила и шаги:

- 1 Каждый игрок рисует свой рисунок по полученной теме.
- 2 Время на рисование ограничено таймером.
- 3 Во время рисования на экране отображается тема рисунка.
- 4 После истечения времени (или при нажатии кнопки "Закончить") рисунок автоматически сохраняется.
- 5 Можно отменить последнее действие на холсте нажав на кнопку стрелки или комбинацию клавиш Ctrl + Z.



Суши-ролл с глазами

Закончить

<-

Нарисуйте это!

Рисунок 6.5 – Окно рисования оригинального рисунка

Правила и задачи в этапе рисований копий многим схожи с рисованием оригиналов, однако все же отличия присутствуют:

- 1 После завершения оригинальных рисунков программа раздаёт каждому игроку чужой рисунок (не его собственный) без указания темы.
- 2 Игрок должен постараться максимально точно скопировать рисунок, ориентируясь только на визуальные элементы.
- 3 Время на копирование также ограничено таймером
- 4 Тема оригинала при этом не отображается — игрок работает вслепую
- 5 Как и в предыдущем этапе можно отменить последнее действие на холсте нажав на кнопку стрелки или комбинацию клавиш Ctrl + Z.

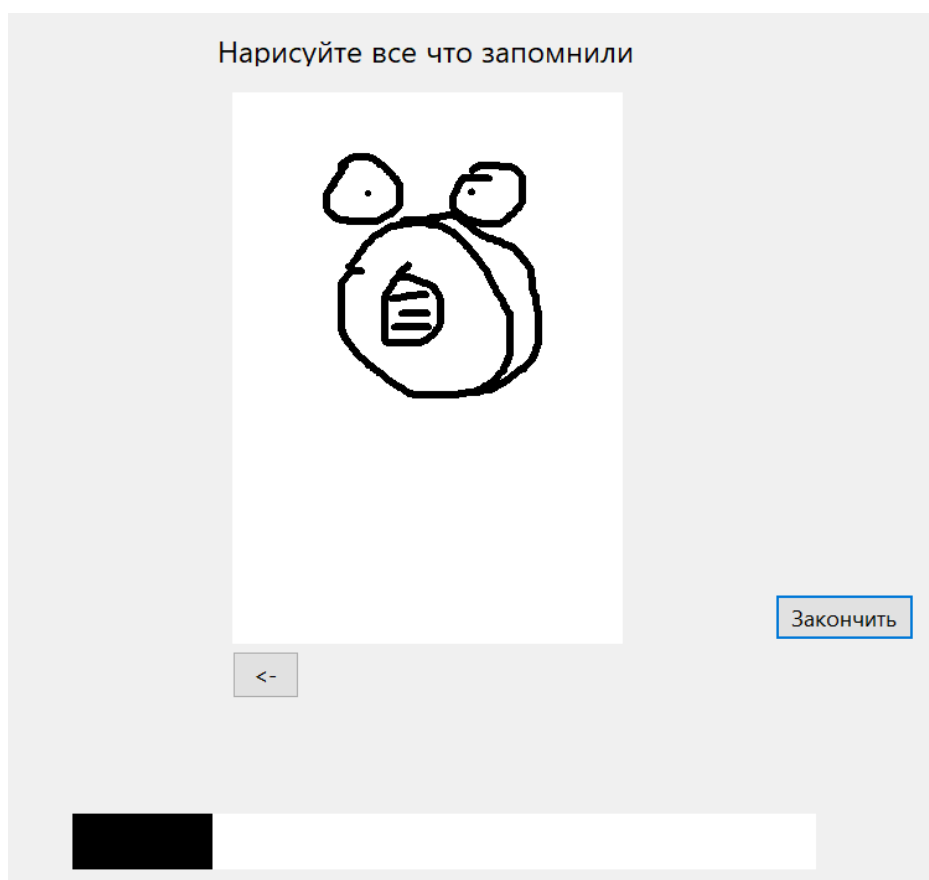


Рисунок 6.6 – Окно рисования копии рисунка

Правила этапа голосования:

- 1 На экране отображаются оригинал одного из игроков и все его копии, в случайном порядке.
- 2 Игроки голосуют, где, по их мнению, оригинальный рисунок. Чтобы проголосовать, игрок должен перетащить карточку с именем на рисунок, который он считает оригинальным.
- 3 Появляются карточки только тех игроков, чьи рисунки на данный момент не отображаются (ни копии, ни оригинал).

4 После окончания пользователями голосования, программа фиксирует выбор и переходит к подсчёту очков.

5 Игрок получает очки за правильное определение оригинала и за то, что его копию приняли за оригинал.

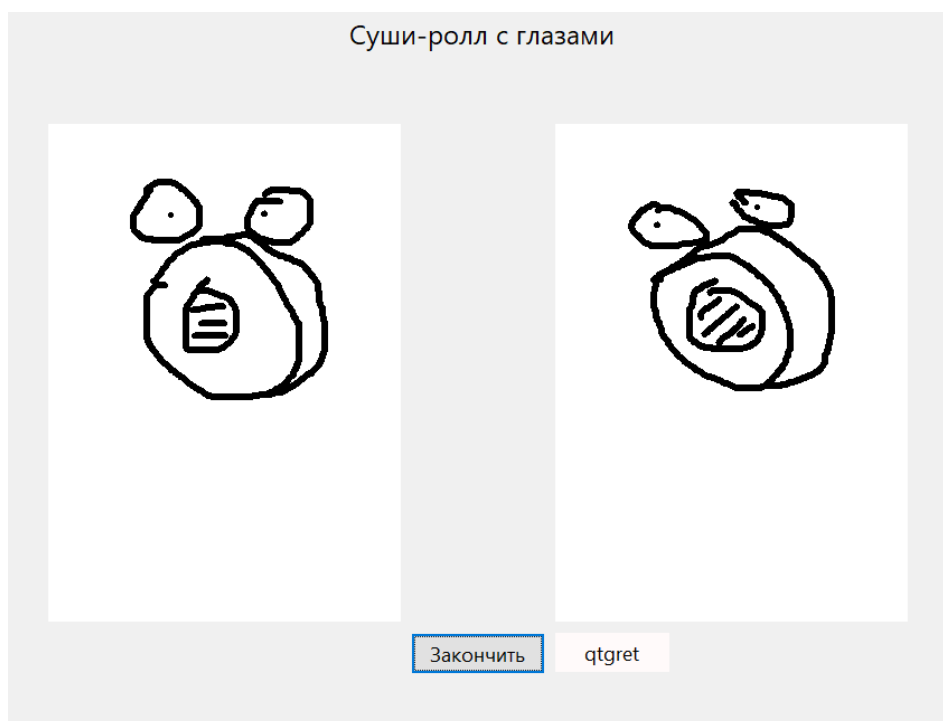


Рисунок 6.7 – Окно голосования за оригинальный рисунок

Самый последний – этап показа результатов и определения победителей:

1 Отображается таблица с псевдонимами игроков и их набранными очками.

2 По количеству очков программа определяет победителей. Если несколько игроков набрали одинаковое количество очков – они считаются победителями.

3 Игрокам предлагается выйти обратно в меню.

Победители выделяются желтым, таблица лидеров отсортирована по количеству очков:

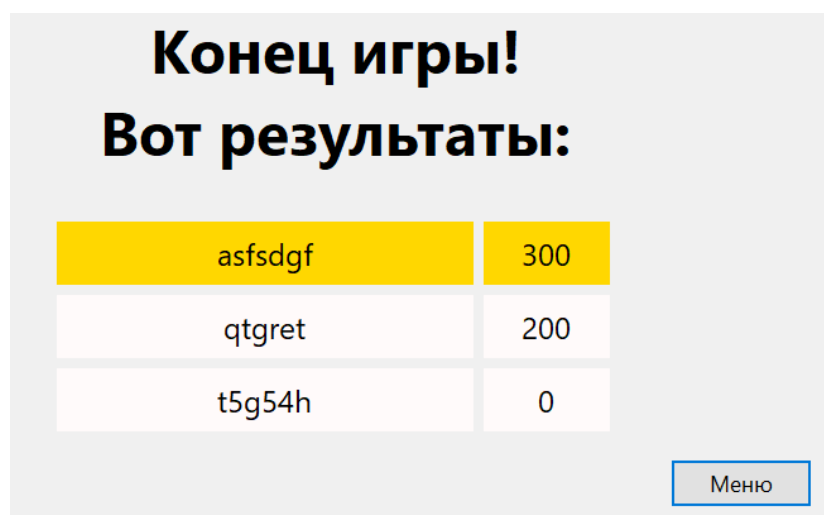


Рисунок 6.8 – Окно результатов игры

При нажатии на кнопку меню все игроки, их рисунки и очки сбрасываются. Следует начать новую игру и заново зарегистрировать всех игроков.

В файле “prompts.txt” хранятся все темы заданий в текстовом виде.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была спроектирована и разработана игра, основанная на механике Copyrighted Artists, адаптированная для использования на одном устройстве без подключения к сети. Программное средство реализовано в среде Delphi VCL, что позволило эффективно организовать интерфейс и реализовать все необходимые этапы игрового процесса – от ввода игроков и генерации заданий до рисования, голосования и подсчета очков.

В процессе работы были выполнены следующие задачи:

1 Проведён анализ аналогичных программных продуктов и сформулированы требования к собственной разработке.

2 Описана функциональность программного средства и построены модели вариантов использования.

3 Спроектированы алгоритмы и структура программы с учетом модульного подхода.

4 Реализован программный код с использованием визуальных и графических компонентов.

5 Проведено всестороннее тестирование, подтверждающее соответствие ПС заявленным требованиям.

Разработка данной программы представляет собой не только реализацию творческого игрового механизма, но и практическую задачу по проектированию, созданию

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

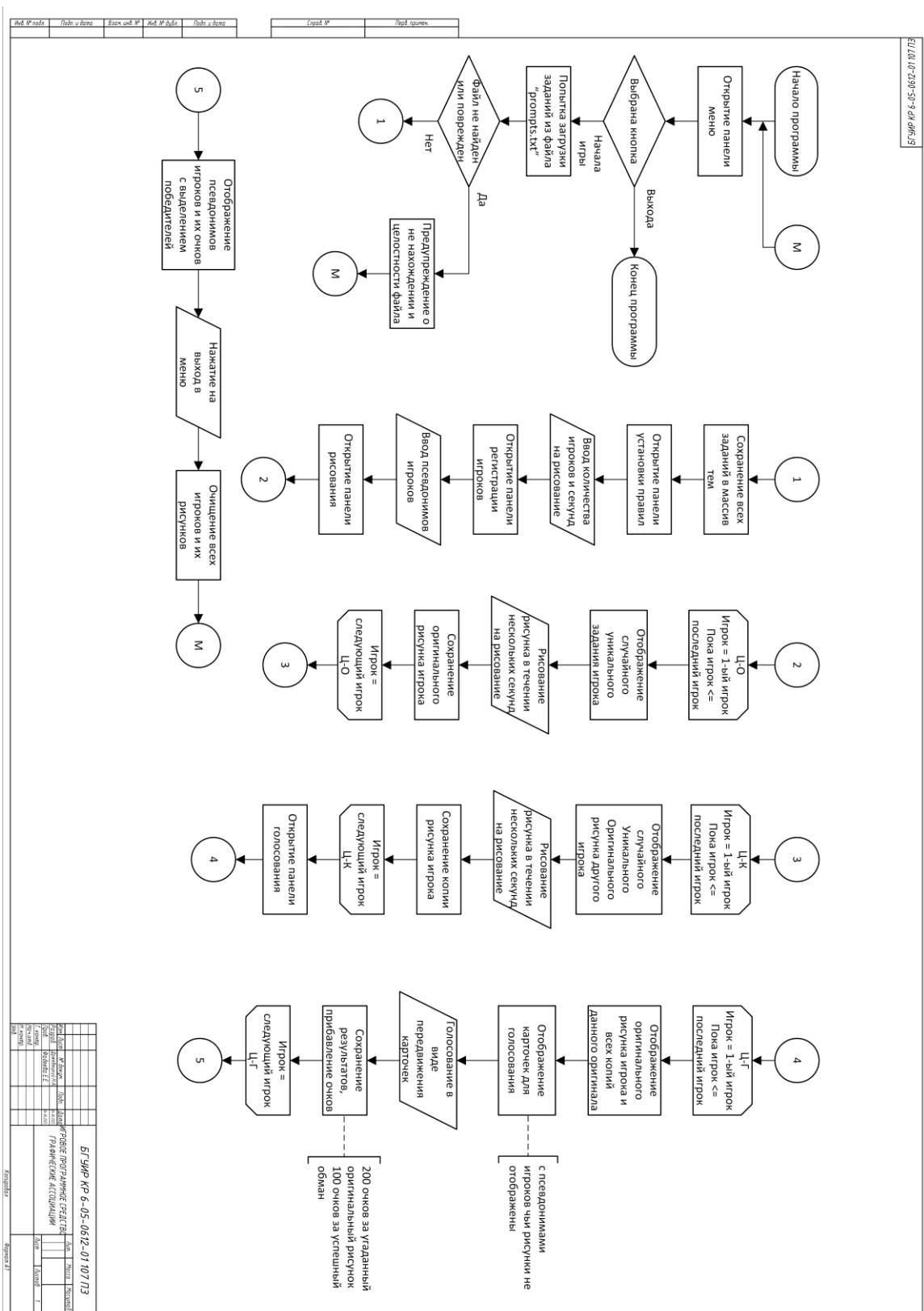
[1] Copyrighted Artists [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.roblox.com/games/4353458311>.

[2] Правила игры Copyrighted Artists [Электронный ресурс]. – Электронные данные. – Режим доступа: https://roblox.fandom.com/wiki/Rawblocky/Copyrighted_Artists.

[3] Правила игры Drawful [Электронный ресурс]. – Электронные данные. – Режим доступа: [https://jackboxgames.fandom.com/wiki/Drawful_\(series\)](https://jackboxgames.fandom.com/wiki/Drawful_(series))

[4] Страница проекта “Battle Of Copies” на платформе GitHub [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/Nik7069/BattleOfCopies/releases/latest>

ПРИЛОЖЕНИЕ А



ПРИЛОЖЕНИЕ Б

Модуль Main

```
unit Main;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants,
  System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs, Vcl.ExtCtrls, Vcl.StdCtrls,
  Vcl.ComCtrls, Vcl.Grids,

  // Мои юниты
  GameLogic, PanelsLogic, SetRulesLogic, WinLogic, VoteLogic;

const
  MinPlayers = 3;
  MaxPlayers = 8;
  MinSeconds = 10;
  MaxSeconds = 120;

type
  TMainForm = class(TForm)
    tGame: TTimer;
    btnBack: TButton;
    btnFinish: TButton;
    pnlGame: TPanel;
    pbDraw: TPaintBox;
    pnlMenu: TPanel;
    btnStart: TButton;
    pnlSetRules: TPanel;
    btnSetRules: TButton;
    ComboPlayers: TComboBox;
    ComboSeconds: TComboBox;
    pnlSetName: TPanel;
    btnSetName: TButton;
    NameInput: TEdit;
    txtCurrentPrompt: TStaticText;
    CustomPB: TPanel;
    ProgressPB: TLabel;
    pnlWaitReady: TPanel;
    btnReady: TButton;
    txtReadyPlayer: TStaticText;
    txtAreReady: TStaticText;
    pnlVote: TPanel;
    txtPromptVote: TStaticText;
    PaintVote1: TPaintBox;
    PaintVote2: TPaintBox;
    btnEndVote: TButton;
    fpVoteUsers: TFlowPanel;
    fpPaintVote1: TFlowPanel;
    fpPaintVote2: TFlowPanel;
    lbPlayersHint: TLabel;
    lbSecondsHint: TLabel;
    lbPlayerNum: TLabel;
    txtCurrentPromptTemp: TStaticText;
    txtPaintHint: TStaticText;
    spnlSetRules: TPanel;
    spnlSetName: TPanel;
    spnlWaitReady: TPanel;
    txtOrigOrCopyPV1: TStaticText;
```



```

txtOrigOrCopyPV2: TStaticText;
pnlWinTable: TPanel;
fpWinTable: TFlowPanel;
txtEndOfGame: TStaticText;
spnlWinTable: TPanel;
txtExitToMenu: TButton;
btnExit: TButton;
procedure pbDrawMouseUp(Sender: TObject; Button: TMouseButton;
  Shift: TShiftState; X, Y: Integer);
procedure pbDrawMouseDown(Sender: TObject; Button: TMouseButton;
  Shift: TShiftState; X, Y: Integer);
procedure pbDrawMouseMove(Sender: TObject; Shift: TShiftState;
  X, Y: Integer);
procedure btnBackClick(Sender: TObject);
procedure btnFinishClick(Sender: TObject);
procedure pbDrawPaint(Sender: TObject);
procedure FormCreate(Sender: TObject);
procedure btnStartClick(Sender: TObject);
procedure btnSetRulesClick(Sender: TObject);
procedure btnSetNameClick(Sender: TObject);
procedure tGameTimer(Sender: TObject);
procedure btnReadyClick(Sender: TObject);
procedure btnShowOrigAndCopyClick(Sender: TObject);
procedure btnCalculatePointsClick(Sender: TObject);
procedure PaintVote1Paint(Sender: TObject);
procedure PaintVote2Paint(Sender: TObject);
procedure ToSelectCardClick(Sender: TObject);
procedure PaintVoteClick(Sender: TObject);
procedure txtExitToMenuClick(Sender: TObject);
procedure FormKeyDown(Sender: TObject; var Key: Word; Shift: TShiftState);
procedure btnExitClick(Sender: TObject);
private
  { Private declarations }
public
  { Public declarations }
end;

var
  MainForm: TMainForm;

implementation

uses PointConverter;

{$R *.dfm}

procedure TMainForm.FormCreate(Sender: TObject);
var
  i: Integer;
begin
  // CreateBitmap;
  Randomize;

  begin
    for i := MinPlayers to MaxPlayers do
      ComboPlayers.Items.Add(IntToStr(i));
  end;
  ComboPlayers.ItemIndex := 0;

  begin
    for i := MinSeconds div 10 to MaxSeconds div 10 do
      ComboSeconds.Items.Add(IntToStr(i * 10));
    end;
  end;
end;

```

```

    ComboSeconds.ItemIndex := 5;

end;

procedure TMainForm.FormKeyDown(Sender: TObject; var Key: Word;
  Shift: TShiftState);
begin
  if (Key = Ord('Z')) and (ssCtrl in Shift) then
  begin
    SetUndoBitmap;
  end;
end;

procedure TMainForm.ToSelectCardClick(Sender: TObject);
begin
  SelectCard(Sender);
end;

procedure TMainForm.txtExitToMenuClick(Sender: TObject);
begin
  ShowPanel(MainForm.pnlMenu);
end;

procedure TMainForm.btnShowOrigAndCopyClick(Sender: TObject);
begin
  ShowOrigAndCopy;
end;

procedure TMainForm.btnCalculatePointsClick(Sender: TObject);
begin
  CalculatePoints;
end;

procedure TMainForm.btnExitClick(Sender: TObject);
begin
  FreePlayersList;
  Close;
end;

procedure TMainForm.btnReadyClick(Sender: TObject);
begin
  PlayerIsReady;
end;

procedure TMainForm.btnStartClick(Sender: TObject);
begin
  LoadPrompts;
end;

procedure TMainForm.btnBackClick(Sender: TObject);
begin
  SetUndoBitmap;
end;

procedure TMainForm.btnFinishClick(Sender: TObject);
begin
  pbTime := -1;
end;

procedure TMainForm.btnSetRulesClick(Sender: TObject);
begin
  AddNewRules;
end;

```

```

procedure TMainForm.btnSetNameClick(Sender: TObject);
begin
    AddNewPlayer;
end;

procedure TMainForm.pbDrawMouseDown(Sender: TObject; Button: TMouseButton;
    Shift: TShiftState; X, Y: Integer);
begin
    PaintMouseDown(X, Y);
end;

procedure TMainForm.pbDrawMouseMove(Sender: TObject; Shift: TShiftState;
    X, Y: Integer);
begin
    PaintMouseMove(X, Y);
end;

procedure TMainForm.pbDrawMouseUp(Sender: TObject; Button: TMouseButton;
    Shift: TShiftState; X, Y: Integer);
begin
    PaintMouseUp;
end;

procedure TMainForm.pbDrawPaint(Sender: TObject);
begin
    LoadPaint;
end;

procedure TMainForm.PaintVoteClick(Sender: TObject);
begin
    PaintVoteSelect(Sender);
end;

procedure TMainForm.PaintVote1Paint(Sender: TObject);
begin
    LoadPaintVote(BallotList[0].PaintVote, BallotList[0].Bitmap);
end;

procedure TMainForm.PaintVote2Paint(Sender: TObject);
begin
    LoadPaintVote(BallotList[1].PaintVote, BallotList[1].Bitmap);
end;

procedure TMainForm.tGameTimer(Sender: TObject);
begin
    GameTimer;
end;

end.

```

Модуль GameLogic

```

unit GameLogic;

interface

uses
    Winapi.Windows, Vcl.Graphics, System.SysUtils;

procedure LoadPaint;
procedure CreateBitmap;
procedure PaintMouseDown(X, Y: Integer);
procedure PaintMouseMove(X, Y: Integer);

```

```

procedure PaintMouseUp;
procedure StartGame;
procedure GameTimer;
procedure PlayerIsReady;
procedure NextPlayerWaitReady;
procedure CleanBitmap(var CleanThisBM: TBitmap);
procedure AfterGame;
procedure HidePaintBox(hide: Boolean);
procedure SetUndoBitmap;

var
    pbTime: Integer;

implementation

uses
    Main, System.Classes, SetRulesLogic, PanelsLogic, VoteLogic;

const
    WaitPromptTimeLength = 3000;
    RememberTimeLength = 5000;
    UndoLimit = 100;

type
    TUndoBuffer = array[0..UndoLimit-1] of TBitmap;

var
    MyColor: TColor;
    Size: Integer = 6;
    MClick: Boolean;
    Bitmap: TBitmap;
    CurrentPlayer: TPlayer;
    CurrentPlayerNum, CurrentOrigNum: Integer;
    pbMax, pbStartTime: Integer;
    IsPaintOriginal: Boolean;
    RoundNum: Integer;
    OnTimerFinish: TProc;

    UndoBuffer: TUndoBuffer;
    UndoIndex: Integer = 0;
    CanUndo: Boolean = false;

procedure LoadPaint;
begin
    MainForm.pbDraw.Canvas.Draw(0, 0, Bitmap);
end;

procedure CleanBitmap(var CleanThisBM: TBitmap);
begin
    CleanThisBM.Canvas.Brush.Color := clWhite;
    CleanThisBM.Canvas.FillRect(Rect(0, 0, CleanThisBM.Width, CleanThisBM.Height));
end;

procedure CreateBitmap;
begin
    Bitmap := TBitmap.Create;
    Bitmap.SetSize(MainForm.pbDraw.Width, MainForm.pbDraw.Height);
    Bitmap.Canvas.Pen.Color := clBlack;
    Bitmap.Canvas.Pen.Width := Size;
end;

procedure SaveUndoBitmap;

```

```

var
  bmp: TBitmap;
begin
  if CanUndo then
  begin
    UndoIndex := (UndoIndex + 1) mod UndoLimit;
    if Assigned(UndoBuffer[UndoIndex]) then
      UndoBuffer[UndoIndex].Free;

    bmp := TBitmap.Create;
    bmp.Assign(Bitmap);

    UndoBuffer[UndoIndex] := bmp;
  end;

end;

procedure SetUndoBitmap;
var
  NextIndex, PrevIndex: Integer;
begin
  if CanUndo then
  begin
    PrevIndex := (UndoIndex - 1 + UndoLimit) mod UndoLimit;

    if Assigned(UndoBuffer[PrevIndex]) then
    begin
      Bitmap.Assign(UndoBuffer[PrevIndex]);
      LoadPaint;

      NextIndex := UndoIndex;
      if Assigned(UndoBuffer[NextIndex]) then
      begin
        UndoBuffer[NextIndex].Free;
        UndoBuffer[NextIndex] := nil;
      end;

      UndoIndex := PrevIndex;
    end;
  end;
end;

procedure PaintMouseDown(X, Y: Integer);
begin
  MClick := true;

  Bitmap.Canvas.MoveTo(X, Y);
  Bitmap.Canvas.LineTo(X, Y);

  LoadPaint;
end;

procedure PaintMouseMove(X, Y: Integer);
begin
  if MClick then
  begin
    Bitmap.Canvas.LineTo(X, Y);
    LoadPaint;
  end;
end;

```

```

procedure PaintMouseUp;
begin
    MClick := false;
    SaveUndoBitmap;
end;

procedure StartTimerPB(Duration: Integer; AfterProc: TProc);
begin
    pbStartTime := GetTickCount;
    pbTime := Duration;

    // Прогресс бар "падает"
    pbMax := MainForm.CustomPB.Width;
    // pbMax - заполнен / 0 - пустой
    MainForm.ProgressPB.Width := pbMax;

    OnTimerFinish := AfterProc;

    MainForm.tGame.Interval := 1; // Любое значение, лучше < 50 для плавности
    MainForm.tGame.Enabled := True;
end;

procedure NextPlayerWaitPrompt(
    const PromptCaption, TempCaption, Hint: string;
    AfterProc: TProc);
begin
    HidePaintBox(true);

    MainForm.txtCurrentPrompt.Caption := PromptCaption;
    MainForm.txtCurrentPromptTemp.Caption := TempCaption;
    MainForm.txtPaintHint.Caption := Hint;

    StartTimerPB(WaitPromptTimeLength, AfterProc);
end;

procedure NextPlayerGame;
begin

    CleanBitmap(Bitmap);
    LoadPaint;

    HidePaintBox(false);

    If IsPaintOriginal then
    begin
        MainForm.txtPaintHint.Caption := 'Нарисуйте это!';
    end
    else
    begin
        MainForm.txtPaintHint.Caption := '';
    end;

    CanUndo := true;
    SaveUndoBitmap;

    StartTimerPB(CountSeconds*1000, AfterGame);

end;

procedure AfterGame;
begin
    if IsPaintOriginal then

```

```

begin
    CurrentPlayer.BitmapOrig.Assign(Bitmap);
end
else
begin
    CurrentPlayer.BitmapCopy.Assign(Bitmap);
end;

CanUndo := false;

Inc(CurrentPlayerNum);
if CurrentPlayerNum = CountPlayers then
begin
    CurrentPlayerNum := 0;
    if not IsPaintOriginal then
    begin
        VoteStart;
        Inc(RoundNum);
    end;

    IsPaintOriginal := not IsPaintOriginal;
end;

if not IsVoting then
begin
    NextPlayerWaitReady;
end
end;

procedure NextPlayerRemember;
begin

    HidePaintBox(false);
    MainForm.btnBack.Visible := false;
    MainForm.btnFinish.Visible := false;

    CurrentOrigNum := (CurrentPlayerNum+1) mod CountPlayers;
    PlayersList[CurrentOrigNum].copyid := CurrentPlayerNum;

    Bitmap.Assign(PlayersList[CurrentOrigNum].BitmapOrig);
    LoadPaint;
    MainForm.pbDraw.Enabled := false;

    StartTimerPB(RememberTimeLength, procedure
    begin
        NextPlayerWaitPrompt(
            'Нарисуйте все что запомнили',
            'Приготовьтесь нарисовать это',
            'Вспомните все детали!',
            NextPlayerGame
        );
    end);
end;

procedure NextPlayerWaitReady;
begin
    ShowPanel(MainForm.pnlWaitReady);
    CurrentPlayer := PlayersList[CurrentPlayerNum];
    MainForm.txtReadyPlayer.Caption := CurrentPlayer.Name;
end;

procedure HidePaintBox(hide: Boolean);
begin

```

```

MainForm.pbDraw.Enabled := not hide;
MainForm.txtCurrentPrompt.Visible := not hide;
MainForm.pbDraw.Visible := not hide;
MainForm.txtCurrentPromptTemp.Visible := hide;

MainForm.btnBack.Visible := not hide;
MainForm.btnFinish.Visible := not hide;
end;

procedure StartGame;
begin
    CreateBitmap;
    CurrentPlayerNum := 0;
    RoundNum := 0;
    IsPaintOriginal := true;
    NextPlayerWaitReady;
end;

procedure PlayerIsReady;
begin
    if IsPaintOriginal then
    begin
        NextPlayerWaitPrompt(
            CurrentPlayer.Prompt,
            CurrentPlayer.Prompt,
            'Приготовьтесь это нарисовать!',
            NextPlayerGame
        );
    end
    else
    begin
        NextPlayerWaitPrompt(
            'Запомните данный рисунок',
            'Приготовьтесь запомнить рисунок',
            'Запомните все детали!',
            NextPlayerRemember
        );
    end;
end;

ShowPanel(MainForm.pnlGame);
end;

procedure GameTimer;
var ElapsedTime: Integer;
begin
    ElapsedTime := GetTickCount - pbStartTime;
    MainForm.ProgressPB.Width := pbMax - Round(pbMax * ElapsedTime / pbTime);

    if ElapsedTime >= pbTime then
    begin
        MainForm.tGame.Enabled := False;

        if Assigned(OnTimerFinish) then
            OnTimerFinish();
        end;
    end;
end.

```

Модуль PanelsLogic

```
unit PanelsLogic;
```



```

interface

uses
  Vcl.ExtCtrls;

procedure ShowPanel(Pannel: TPanel);

implementation

uses Main;

procedure ShowPanel(Pannel: TPanel);
begin
  // All panels
  MainForm.pnlGame.Visible := False;
  MainForm.pnlMenu.Visible := False;
  MainForm.pnlSetRules.Visible := False;
  MainForm.pnlSetNames.Visible := False;
  MainForm.pnlWaitReady.Visible := False;
  MainForm.pnlVote.Visible := False;
  MainForm.pnlWinTable.Visible := False;

  Pannel.Visible := True;
  Pannel.BringToFront;
end;

end.

```

Модуль SetRulesLogic

```

unit SetRulesLogic;

interface

uses
  Vcl.Graphics;

procedure AddNewPlayer;
procedure AddNewRules;
procedure LoadPrompts;

type
  TPlayer = Record
    Name: String;
    Prompt: String;
    id, copyid: Integer;
    Score: Integer;
    BitmapOrig, BitmapCopy: TBitmap;
  End;

  type
    TPlayersList = array of TPlayer;
    TPromptsList = array of String;

  var
    PlayersList: TPlayersList;
    PromptsList: TPromptsList;
    CountPlayers, CountSeconds: Integer;

implementation

uses
  Main, PanelsLogic, WinLogic, VoteLogic, GameLogic, System.SysUtils,
  Vcl.Dialogs;

```

```

var
  iName: Integer;

procedure ShuffleFirstN(var SomeList: TPromptsList; N: Integer);
var
  i, r: Integer;
  temp: String;
begin
  for i := 0 to N - 1 do
    begin
      r := Random(Length(SomeList) - i) + i;
      temp := SomeList[i];
      SomeList[i] := SomeList[r];
      SomeList[r] := temp;
    end;
  end;

procedure AddNewRules;
var
  IndexPlayers: Integer;
begin
  IndexPlayers := MainForm.ComboPlayers.ItemIndex;
  CountSeconds := StrToIntDef(MainForm.ComboSeconds.Text, -1);

  if CountPlayers > Length(PromptsList) then
    begin
      ShowMessage
        ('Игроков больше чем уникальных заданий. Добавьте больше тем или уменьшите кол-во игроков');
    end
  else if IndexPlayers = -1 then
    begin
      ShowMessage('Вы ввели неверное количество игроков, их должно быть от ' +
        IntToStr(MinPlayers) + ' до ' + IntToStr(MaxPlayers));
    end
  else if (CountSeconds < MinSeconds) or (CountSeconds > MaxSeconds) then
    begin
      ShowMessage('Вы ввели неверное количество секунд, их должно быть от ' +
        IntToStr(MinSeconds) + ' до ' + IntToStr(MaxSeconds));
    end
  else
    begin
      CountPlayers := StrToInt(MainForm.ComboPlayers.Items
        [MainForm.ComboPlayers.ItemIndex]);

      SetLength(PlayersList, CountPlayers);
      ShuffleFirstN(PromptsList, CountPlayers);
      MainForm.IbPlayerNum.Caption := 'Игрок № [1/' +
        IntToStr(CountPlayers) + ']';

      iName := 0;
      ShowPanel(MainForm.pnlSetNames);
    end;

end;

procedure AddNewPlayer;
var
  NewPlayer: TPlayer;
  i: Integer;
  SameName: Boolean;
begin
  NewPlayer.Name := Trim(MainForm.NameInput.Text);

```

```

SameName := false;
// for i := Low(PlayersList) to iName-1 do
// begin
// if PlayersList[i].Name = NewPlayer.Name then
// SameName := true;
// end;

i := Low(PlayersList);
while (not SameName) and (i < iName) do
begin
  if PlayersList[i].Name = NewPlayer.Name then
    SameName := true;
    Inc(i);
end;

if (not SameName) and (NewPlayer.Name <> "") then
begin
  NewPlayer.id := iName;
  NewPlayer.Prompt := PromptsList[iName];
  NewPlayer.BitmapOrig := TBitmap.Create;
  NewPlayer.BitmapCopy := TBitmap.Create;
  NewPlayer.Score := 0;
  PlayersList[iName] := NewPlayer;
  MainForm.NameInput.Clear;

  Inc(iName);

  MainForm.lbPlayerNum.Caption := 'Игрок № [' + IntToStr(iName + 1) + ']' +
    IntToStr(CountPlayers) + ']';

  if iName = CountPlayers then
  begin
    iName := 0;
    // VoteStart;
    StartGame;
    // ShowWinTable;
  end;
end
else
begin
  ShowMessage
    ('Вы ввели уже используемое/некорректное имя. Введите другой псевдоним');
end;

end;

procedure LoadPrompts;
var
  myFile: TextFile;
  Prompt: String;
  OldLength: Integer;
begin
  try
    AssignFile(myFile, 'prompts\prompts.txt');
    Reset(myFile);
    while (not EOF(myFile)) do
    begin
      Readln(myFile, Prompt);
      Prompt := Trim(UTF8ToString(Prompt));
      if Prompt <> "" then
      begin
        OldLength := Length(PromptsList);

```

```

        SetLength(PromptsList, OldLength + 1);
        PromptsList[OldLength] := Prompt;
    end;

    end;
    closeFile(myFile);
    ShowPanel(MainForm.pnlSetRules);
except
    ShowMessage('Файл с заданиями не найден, переустановите программу');
end;

end;

end.

```

Модуль VoteLogic

```

unit VoteLogic;

interface

uses
    Vcl.StdCtrls, SetRulesLogic, Vcl.Graphics, Vcl.ExtCtrls;

procedure VoteStart;
procedure CalculatePoints;
procedure LoadPaintVote(PaintVote: TPaintBox; Bitmap: TBitmap);
procedure SelectCard(Sender: TObject);
procedure PaintVoteSelect(Sender: TObject);
procedure ShowOrigAndCopy;

const
    UnSlctColor = clWebSnow;
    SlctColor = clSkyBlue;
    GuessedOriginalScore = 200;
    GuessedCopyScore = 100;

type
    PPlayer = ^TPlayer;

    TCard = record
        Player: PPlayer;
        LabelCard: TLabel;
        SelectedId: Integer;
    end;

    TBallot = record
        Bitmap: TBitmap;
        PaintVote: TPaintBox;
        Player: PPlayer;
        OrigOrCopyText: TStaticText;
    end;

    TVoteCardsList = array of TCard;

var
    IsVoting: boolean = false;
    VoteCardsList: TVoteCardsList;
    SelectedCard: TLabel;
    BallotList: array of TBallot;

implementation

uses

```

```

Main, System.Classes, Vcl.Controls, Vcl.Dialogs, System.SysUtils,
PanelsLogic, GameLogic, WinLogic;

var
  OriginalNum: Integer;
  VoteOfs, CountBallots: Integer;

procedure LoadPaintVote(PaintVote: TPaintBox; Bitmap: TBitmap);
begin
  PaintVote.Canvas.Draw(0, 0, Bitmap);
end;

procedure CreateNewCard(var Player: TPlayer; VotersCount: Integer);
var
  NewCard: TCard;
begin
  NewCard.LabelCard := TLabel.Create(MainForm);

  // Label
  with NewCard.LabelCard do
  begin
    Parent := MainForm.fpVoteUsers;
    Caption := Player.Name;
    Width := 102;
    Height := 35;
    Color := UnSlctColor;
    Transparent := false;
    Alignment := taCenter;
    Layout := tlCenter;
    AlignWithMargins := true;
    OnClick := MainForm.ToSelectCardClick;
  end;

  // Margins
  with NewCard.LabelCard.Margins do
  begin
    Top := 0;
    Left := 0;
    Bottom := 4;
    Right := 4;
  end;

  NewCard.SelectedId := -1;

  NewCard.Player := @Player;

  VoteCardsList[VotersCount] := NewCard;
end;

procedure SelectCard(Sender: TObject);
var
  Card: TLabel;
begin
  if IsVoting then
  begin
    if Assigned(SelectedCard) then
      SelectedCard.Color := UnSlctColor;

    Card := Sender as TLabel;
    Card.Color := SlctColor;
    SelectedCard := Card;
  end;
end;

```

```

procedure PaintVoteSelect(Sender: TObject);
var
  PaintVote: TPaintBox;
  i, j: Integer;
begin
  if Assigned(SelectedCard) and IsVoting then
  begin
    PaintVote := Sender as TPaintBox;

    i := 0;
    while (i <= 1) and (BallotList[i].PaintVote <> PaintVote) do
      Inc(i);

    if i <= 1 then
    begin
      j := 0;
      while (j <= High(VoteCardsList)) and
        (VoteCardsList[j].LabelCard <> SelectedCard) do
        Inc(j);

      if j <= High(VoteCardsList) then
      begin
        VoteCardsList[j].SelectedId := i;
        VoteCardsList[j].LabelCard.Color := UnSlctColor;
        if i = 0 then

          VoteCardsList[j].LabelCard.Parent := MainForm.fpPaintVote1
        else
          VoteCardsList[j].LabelCard.Parent := MainForm.fpPaintVote2;
        end;
      end;
    end;

  end;
end;

procedure NextPlayerVote;
var
  i: Integer;
  VotersCount: Integer;
  OriginalPlayer: PPlayer;
begin
  IsVoting := true;

  OriginalPlayer := @PlayersList[OriginalNum];
  // CurrentOriginal.copyid := (OriginalNum+1) mod CountPlayers;
  MainForm.txtPromptVote.Caption := OriginalPlayer.Prompt;

  VoteOfs := Random(CountBallots);

  BallotList[VoteOfs].Player := @PlayersList[OriginalNum];
  BallotList[(VoteOfs + 1) mod CountBallots].Player :=
    @PlayersList[OriginalPlayer.copyid];

  BallotList[VoteOfs].Bitmap.Assign(PlayersList[OriginalNum].BitmapOrig);
  BallotList[(VoteOfs + 1) mod CountBallots].Bitmap.Assign
    (PlayersList[OriginalPlayer.copyid].BitmapCopy);

  for i := Low(BallotList) to High(BallotList) do
  begin
    BallotList[i].OrigOrCopyText.Visible := false;
  end;
end;

```

```

LoadPaintVote(BallotList[0].PaintVote, BallotList[0].Bitmap);
LoadPaintVote(BallotList[1].PaintVote, BallotList[1].Bitmap);

SetLength(VoteCardsList, CountPlayers - CountBallots);

VotersCount := 0;

for i := 0 to CountPlayers - 1 do
begin
  if (OriginalNum <> i) and (OriginalPlayer.copyid <> i) then
  begin
    CreateNewCard(PlayersList[i], VotersCount);
    Inc(VotersCount);
  end;
end;

end;

procedure VoteStart;
begin
  CountBallots := 2;
  SetLength(BallotList, CountBallots);

  BallotList[0].PaintVote := MainForm.PaintVote1;
  BallotList[1].PaintVote := MainForm.PaintVote2;

  BallotList[0].Bitmap := TBitmap.Create;
  BallotList[1].Bitmap := TBitmap.Create;

  BallotList[0].OrigOrCopyText := MainForm.txtOrigOrCopyPV1;
  BallotList[1].OrigOrCopyText := MainForm.txtOrigOrCopyPV2;

  OriginalNum := 0;
  ShowPanel(MainForm.pnlVote);
  NextPlayerVote;
end;

procedure ShowOrigAndCopy;
var
  i: Integer;
begin
  for i := Low(BallotList) to High(BallotList) do
  begin
    if i = VoteOfs then
    begin
      BallotList[i].OrigOrCopyText.Caption := 'Оригинал'
    end
    else
    begin
      BallotList[i].OrigOrCopyText.Caption := 'Копия'
    end;

    BallotList[i].OrigOrCopyText.Visible := true;
  end;

  IsVoting := false;

  MainForm.btnEndVote.OnClick := MainForm.btnCalculatePointsClick;
  MainForm.btnEndVote.Caption := 'Продолжить';

end;

procedure CalculatePoints;

```

```

var
  i: Integer;
  Card: TCard;
begin

  MainForm.btnEndVote.OnClick := MainForm.btnShowOrigAndCopyClick;
  MainForm.btnEndVote.Caption := 'Закончить';

  for i := 0 to High(VoteCardsList) do
  begin
    Card := VoteCardsList[i];
    if Card.SelectedId > -1 then
    begin
      if Card.SelectedId = VoteOfs then
      begin
        Card.Player.Score := Card.Player.Score + GuessedOriginalScore;
      end
      else
      begin
        BallotList[Card.SelectedId].Player.Score := BallotList[Card.SelectedId]
          .Player.Score + GuessedCopyScore;
      end;
    end;
  end;

end;

// Очищаем
for i := 0 to High(VoteCardsList) do
begin
  if Assigned(VoteCardsList[i].LabelCard) then
  begin
    VoteCardsList[i].LabelCard.Free;
    VoteCardsList[i].LabelCard := nil;
  end;
end;

Inc(OriginalNum);
if OriginalNum = CountPlayers then
begin
  ShowWinTable;
end
else
begin
  NextPlayerVote;
end;
end;

end.

```

Модуль WinLogic

```

unit WinLogic;

interface

uses
  SetRulesLogic, System.Classes, PanelsLogic, System.SysUtils, Vcl.Graphics,
  Vcl.StdCtrls;

procedure ShowWinTable;
procedure FreePlayersList;

implementation

```



```

uses Main;

const
  StandartColor = clWebSnow;
  WinColor = clWebGold;

procedure SortTopPlayers;
var
  i, j: Integer;
  Temp: TPlayer;
begin
  for i := 0 to High(PlayersList) - 1 do
    for j := i + 1 to High(PlayersList) do
      if PlayersList[j].Score > PlayersList[i].Score then
        begin
          Temp := PlayersList[i];
          PlayersList[i] := PlayersList[j];
          PlayersList[j] := Temp;
        end;
      end;
    end;
  end;

  // function GetTopPlayers(Players: TPlayersList): TPlayersList;
  // var
  //   MaxScore: Integer;
  //   i, len: Integer;
  // begin
  //   // len := Length(Players);
  //   // if len > 1 then
  //   // begin
  //   //   MaxScore := Players[0].Score;
  //   //   // i := 0;
  //   //   while (i < len) and (Players[i].Score = MaxScore) do
  //   //   begin
  //   //   SetLength(Result, i+1);
  //   //   Result[i] := Players[i];
  //   //   Inc(i);
  //   //   end;
  //   //   end;
  //   // end;
  // end;

  procedure FreePlayersList;
  var
    i: Integer;
  begin
    for i := 0 to High(PlayersList) do
      begin
        if Assigned(PlayersList[i].BitmapOrig) then
          PlayersList[i].BitmapOrig.Free;
        if Assigned(PlayersList[i].BitmapCopy) then
          PlayersList[i].BitmapCopy.Free;
        end;
      end;

      SetLength(PlayersList, 0); // очищаем сам массив
      CountPlayers := 0;
    end;

  procedure ShowWinTable;
  var
    i: Integer;

```

```

lblPlayer, lblScore: TLabel;
MaxScore: Integer;
begin

for i := MainForm.fpWinTable.ControlCount - 1 downto 0 do
  if MainForm.fpWinTable.Controls[i] is TLabel then
    MainForm.fpWinTable.Controls[i].Free;

ShowPanel(MainForm.pnlWinTable);

SortTopPlayers;
MaxScore := PlayersList[0].Score;

for i := 0 to High(PlayersList) do
begin
  lblPlayer := TLabel.Create(MainForm);
  lblScore := TLabel.Create(MainForm);

  with lblPlayer do
  begin
    Parent := MainForm.fpWinTable;
    Caption := PlayersList[i].Name;
    Autosize := false;
    Width := 330;
    Height := 50;
    Font.Size := 12;
    Alignment := taCenter;
    Layout := tlCenter;
    Transparent := false;
    AlignWithMargins := true;

    with Margins do
    begin
      Top := 4;
      Left := 4;
      Bottom := 4;
      Right := 4;
    end;
  end;

  with lblScore do
  begin
    Parent := MainForm.fpWinTable;
    Caption := IntToStr(PlayersList[i].Score);
    Autosize := false;
    Width := 100;
    Height := 50;
    Font.Size := 12;
    Alignment := taCenter;
    Layout := tlCenter;
    Transparent := false;
    AlignWithMargins := true;

    with Margins do
    begin
      Top := 4;
      Left := 4;
      Bottom := 4;
      Right := 4;
    end;
  end;

  if PlayersList[i].Score = MaxScore then

```

```
begin
  lblPlayer.Color := WinColor;
  lblScore.Color := WinColor;
end
else
begin
  lblPlayer.Color := StandartColor;
  lblScore.Color := StandartColor;
end;

end;

FreePlayersList;

end;

end.
```

ВЕДОМОСТЬ

Обозначение					Наименование					Дополнительные сведения		
					<u>Текстовые документы</u>							
БГУИР КР 6-05-0612-01 107 ПЗ					Пояснительная записка					62 с.		
					<u>Графические документы</u>							
ГУИР 451001 107 СП					Игровое программное средство графические ассоциации.					Формат А1		